

1.SGLang 的 RadixAttention / RadixCache 解决什么问题？

RadixAttention 的核心目标是自动检测并复用跨请求的公共前缀 KV Cache，从而显著减少重复计算，提升吞吐量并降低内存占用。

传统推理系统在请求结束后会直接丢弃其 KV Cache。而 RadixAttention 将 KV Cache 组织成一棵基数树，一种压缩前缀树。当新请求到来时，系统会沿着树结构查找最长的公共前缀。如果找到匹配项，这部分前缀对应的 KV Cache 就会被直接复用，无需重新计算。多个请求共享相同的系统提示词时，该提示词只需处理一次。树中不再使用的节点会通过 LRU 策略被驱逐，以释放内存。

2.SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

page-sized KV cache→pipeline 中的 KV Cache/RadixCache 这一层。

它在 tokenizer 产出 token 序列之后生效：序列被按 page 切分，Scheduler 以 page 为最小单位，从显存池分配物理块，Prefill 阶段写入 Decode 阶段逐步追加。它的粒度直接决定了 Scheduler 能否把并发 batch 塞满——这是存储层对调度层的约束。

prefix reuse → pipeline 中的 Scheduler/Queue+Prefill 两段。

tokenize 之后，Scheduler 先做基数树前缀匹配（这是调度决策的一部分），然后把“命中长度”作为输入交给 Prefill，Prefill 只对未命中后缀做前向计算。

3.vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异？

vLLM 的 PagedAttention 主要解决的是 KV Cache 的内存碎片化和利用率低下问题。

PagedAttention 借鉴了操作系统虚拟内存的分页机制，将 KV Cache 分割成固定大小的块，并允许这些块存储在非连续的物理内存中。系统通过一个块表来维护逻辑块到物理块的映射，从而实现按需分配，极大减少了内存碎片。

差异：

维度	vLLM PagedAttention	SGLang RadixAttention
核心贡献	KV Cache 的显存管理	跨请求自动前缀复用 + cache-aware 调度
数据结构	哈希表	基数树
匹配粒度	block 粒度，前缀必是 block 大小的整数倍才命中	任意 token 粒度，可命中到 block 中间
分支结构	表达力弱	天然表达树状/分支
调度策略	continuous batching, FCFS 为主	continuous batching + LPM 的 cache-aware 调度
淘汰机制	LRU	LRU
吞吐量	吞吐 2-4×	吞吐最高 6.4×